

Programação em Python

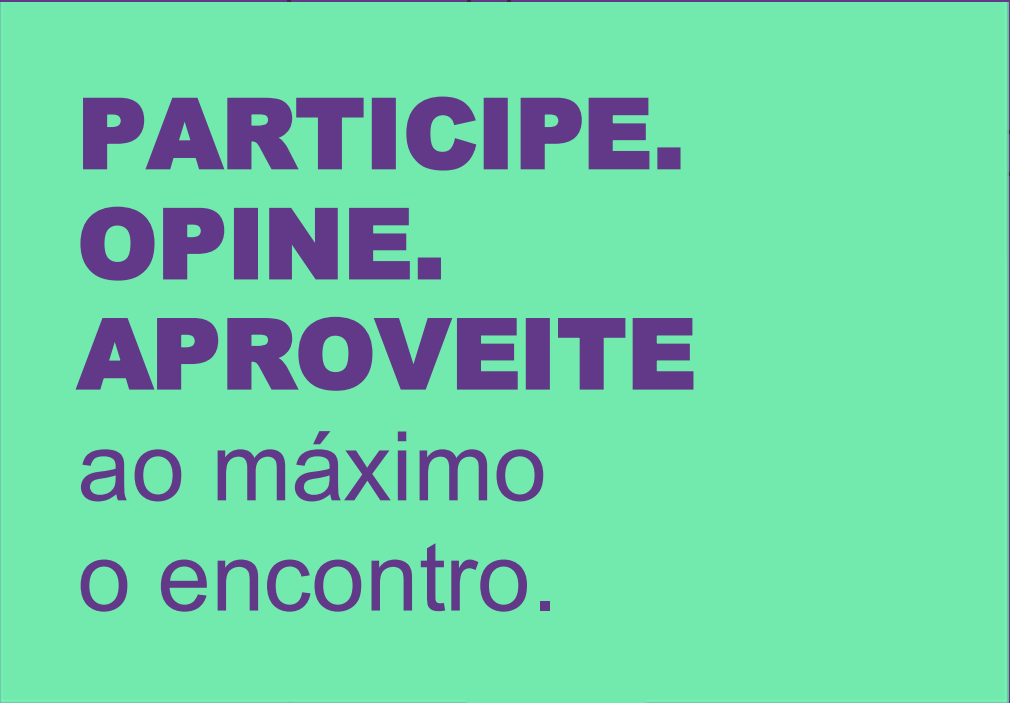
Desenvolver Aplicações Back- End para Web com Python



Bem-vindos à Aula: Desenvolver **Aplicações** **Back-End** para web com **python**

O que faremos hoje:

- Entender o que é um framework
- Conhecer o django
- MVT
- Instalar o django
- Ola mundo!



**PARTICIPE.
OPINE.
APROVEITE**
ao máximo
o encontro.

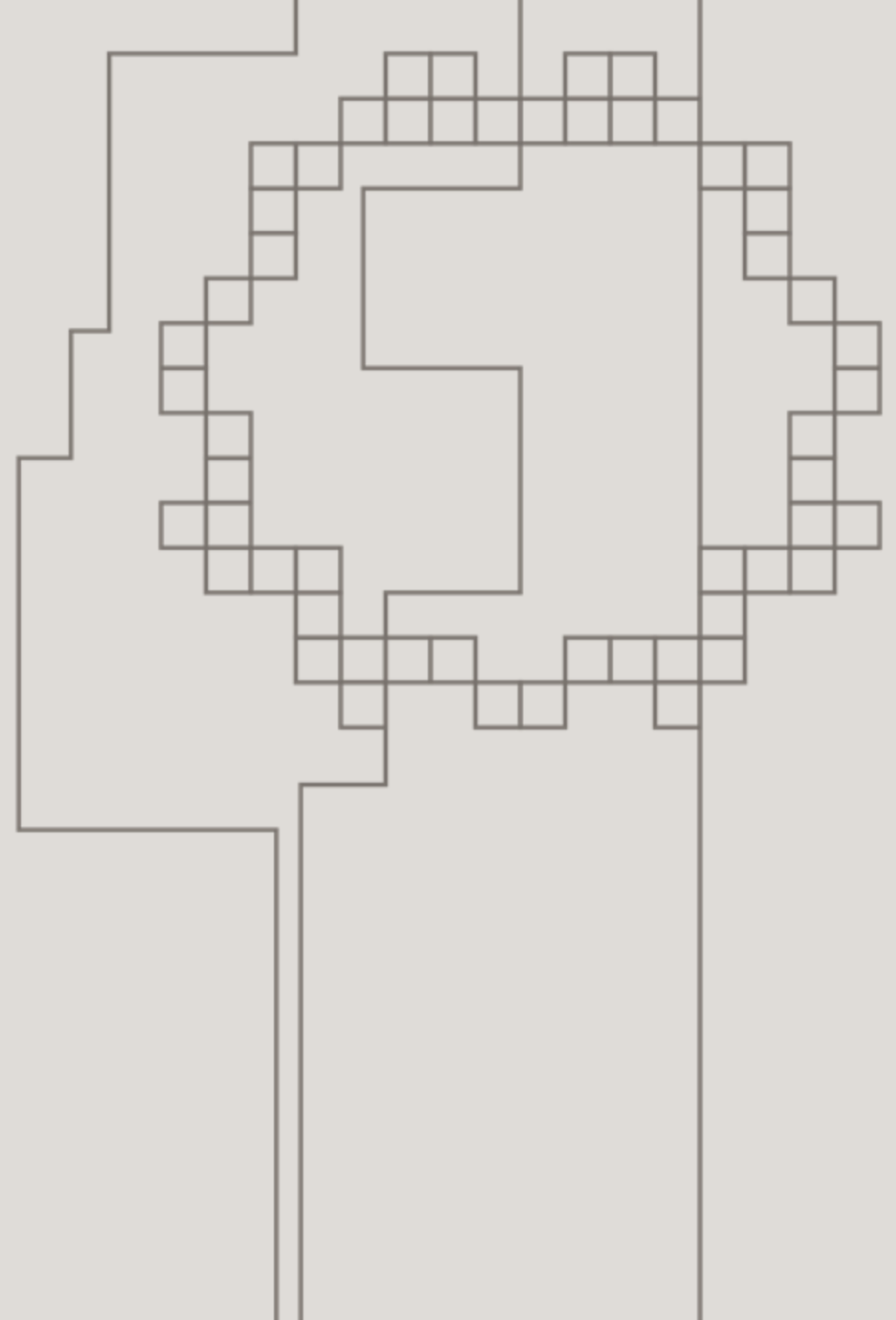
Estamos em um ambiente
heterogêneo e diverso,
onde sempre há respeito.

Então, lembre-se de que:

**NÃO EXISTE
RESPOSTA ERRADA,
PERGUNTA BOBA**
ou que não valha a pena.

Para não termos interrupções e em respeito aos colegas,
pedimos que não faça uso de celular durante a aula e que
o coloque no modo silencioso.

Introdução ao Django





Você sabe me dizer o que é
um framework?



Um framework é uma estrutura de software predefinida que oferece um conjunto de ferramentas e componentes reutilizáveis para o desenvolvimento de aplicações. Ele fornece uma base sobre a qual os desenvolvedores podem construir sistemas mais complexos de forma mais eficiente e padronizada.

Imagine um framework como uma caixa de ferramentas





Faroeste Django



O que é um django?

Django é um framework web de alto nível em Python

Criado para facilitar o desenvolvimento rápido e com design limpo e pragmático





Principais características:

1. **Rápido desenvolvimento.**
2. **DRY (Don't Repeat Yourself).**
3. **Alta segurança.**

Quando usar Django

- Aplicações web robustas e escaláveis.
- Projetos que necessitam de rapidez no desenvolvimento.





Aplicações que usam Django





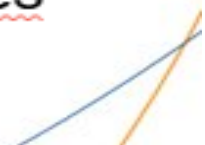


Motivo do Uso de Django: Django proporciona uma base sólida para lidar com o grande volume de dados e tráfego, oferecendo uma estrutura organizada e segura.

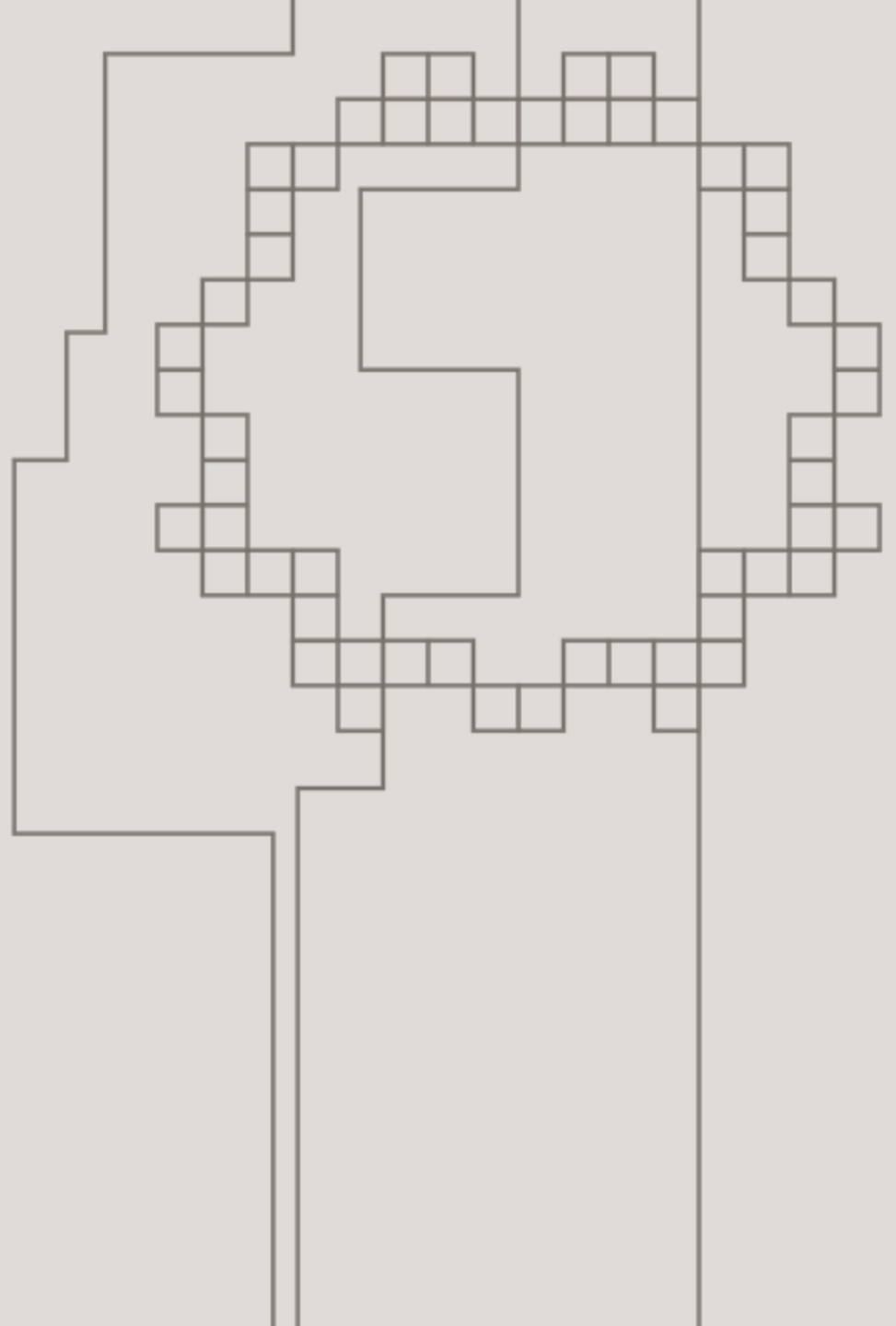
- **Motivo do Uso de Django:** Django facilita a construção de funcionalidades complexas e escaláveis, essenciais para um grande número de usuários ativos e conteúdo visual.



Motivo do Uso de Django: A flexibilidade e segurança do Django permitem a Mozilla desenvolver rapidamente soluções personalizadas e seguras para suas necessidades internas.



MVT - Django





django

MVT é a sigla para **Model-View-Template**, que é o padrão arquitetural usado pelo Django para organizar o desenvolvimento de aplicações web. Vamos entender cada um desses componentes de forma simples e didática:



DJANGO MVT FIELD
MODEL 2-1 TEMPLATE
85:00

django

python

MVT FIELD

django

python

django

MVT

MVT

M

V - VIEW

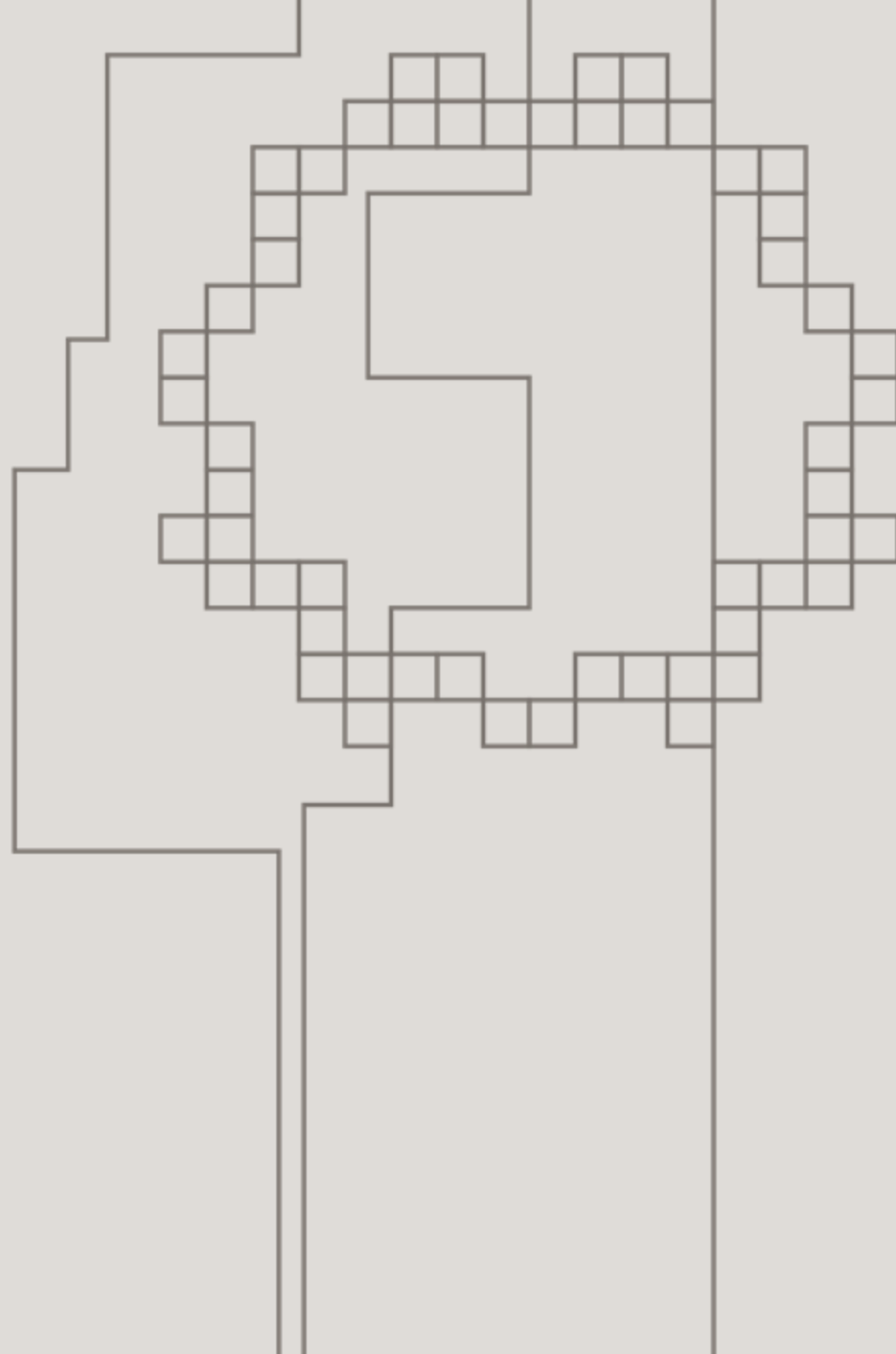
T - TEMPLATE

DEFESA - M DE MODEL
(DADOS)

MEIO DE CAMPO - V DE VIEW
(LÓGICA)

ATAQUE - T DE TEMPLATE
(APRESENTAÇÃO)

Model - Django



O **Model** é responsável por definir a estrutura dos dados e as regras de negócio da aplicação. Ele mapeia as tabelas do banco de dados para classes Python.

Exemplo Simples: Imagine que estamos criando um sistema de blog. Vamos definir um modelo para as postagens do blog.

python

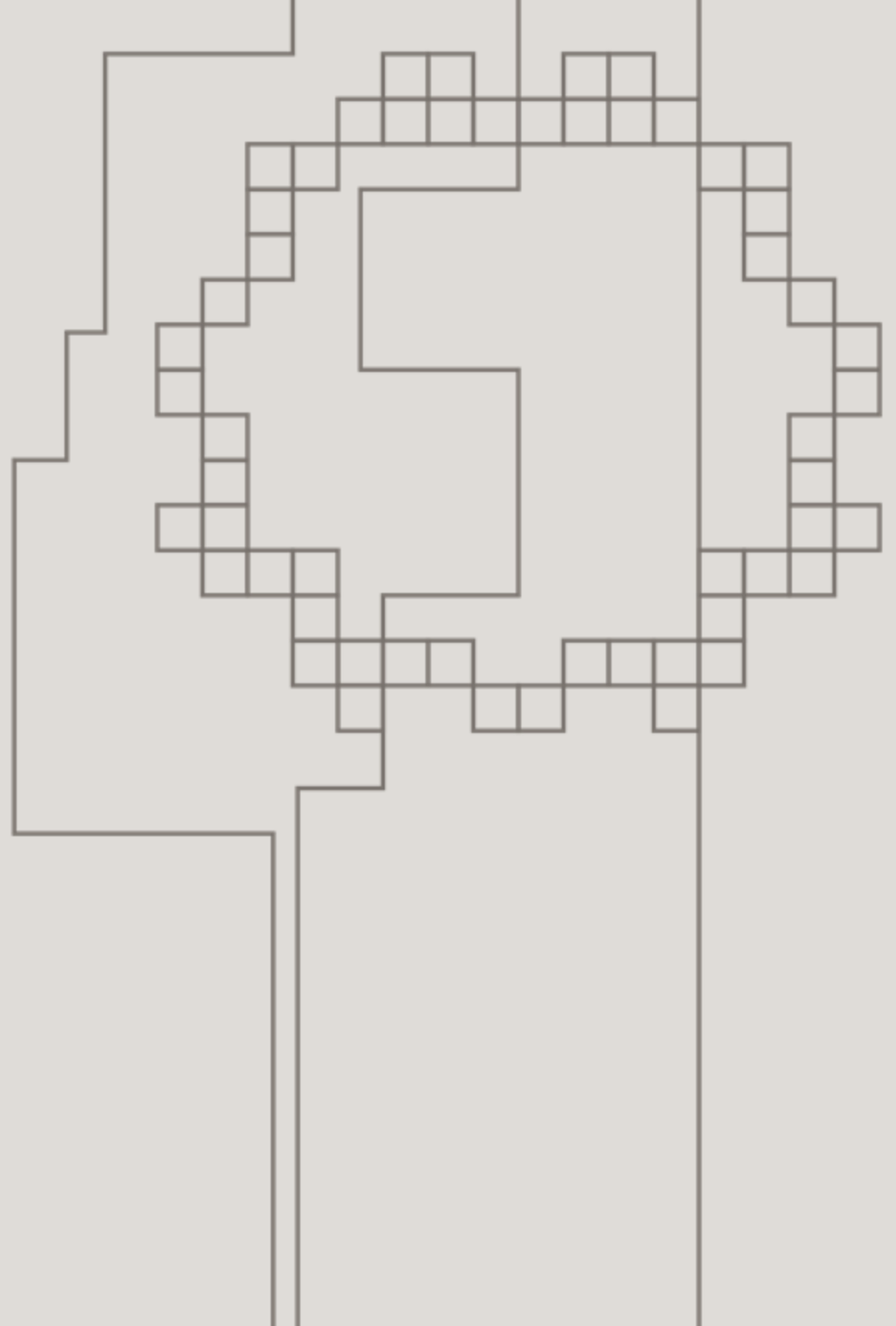
Copiar código

```
# models.py
from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=100)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
```

Aqui, o modelo Post tem três campos: title (título da postagem), content (conteúdo da postagem) e created_at (data e hora de criação da postagem).

View - Django



A **View** é responsável por lidar com a lógica da aplicação. Ela recebe as requisições dos usuários, processa os dados usando os modelos, e retorna uma resposta.

Exemplo Simples: Vamos criar uma view que exibe todas as postagens do blog.

python

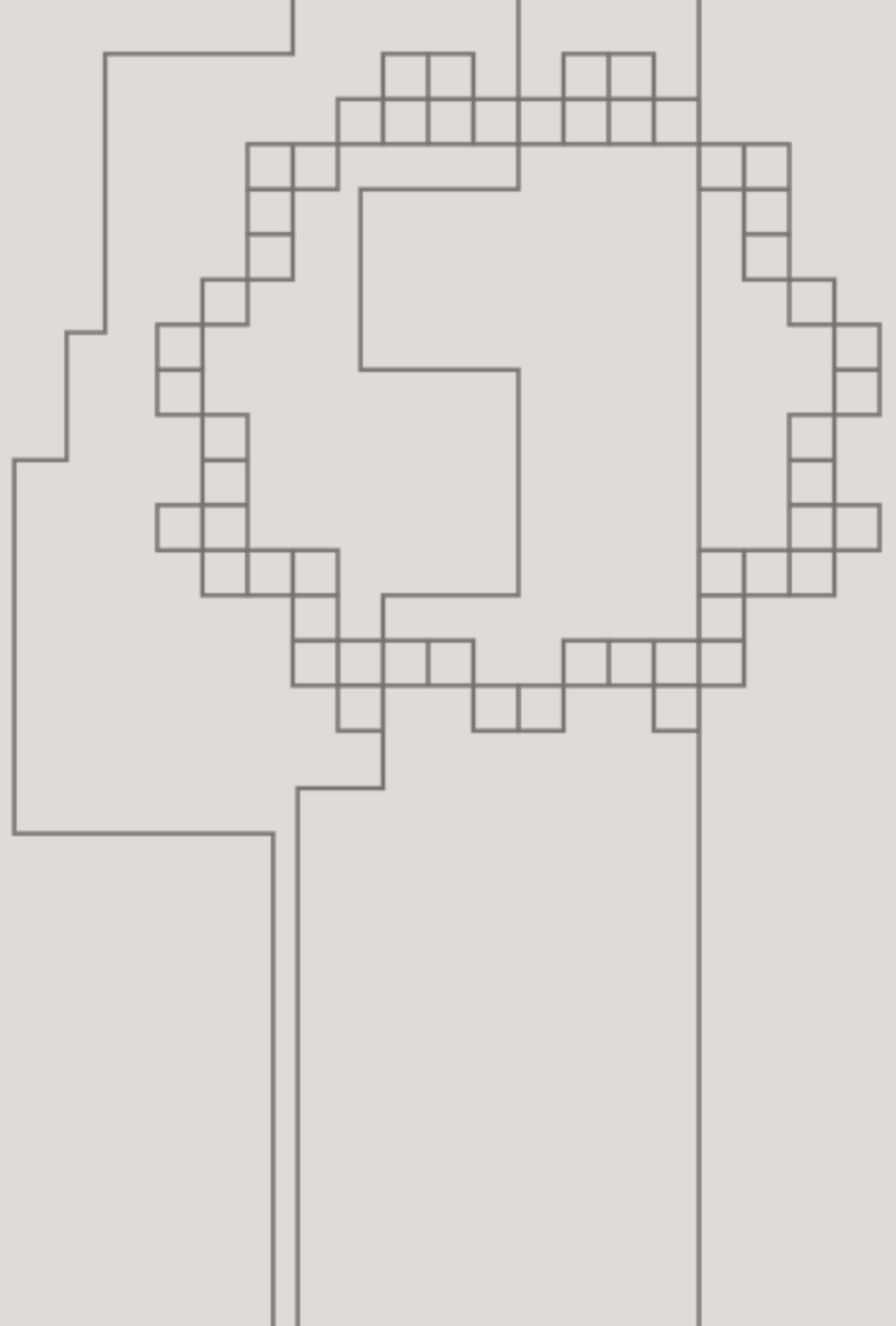
 Copiar código

```
# views.py
from django.shortcuts import render
from .models import Post

def post_list(request):
    posts = Post.objects.all()
    return render(request, 'post_list.html', {'posts': posts})
```

Aqui, a view post_list consulta todas as postagens do banco de dados e as passa para um template chamado post_list.html.

Template - Django



O **Template** é responsável por exibir os dados para os usuários. Ele define a estrutura e o design da página web.

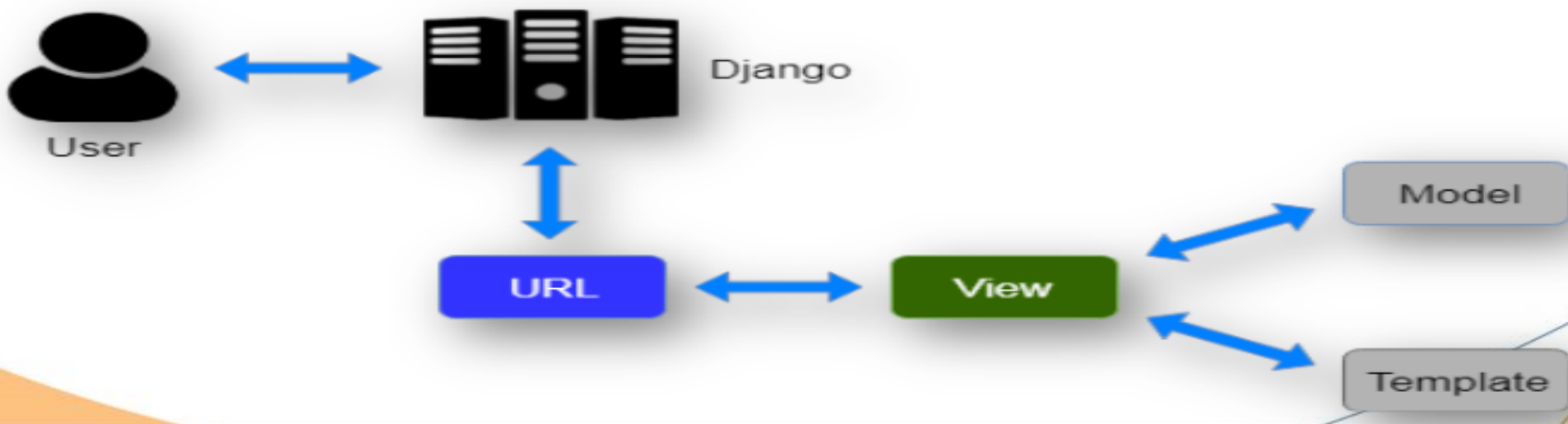
Exemplo Simples: Vamos criar um template que exibe as postagens do blog.

Aqui, o template post_list.html exibe o título, conteúdo e data de criação de cada postagem.

```
<!-- post_list.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Blog</title>
</head>
<body>
  <h1>Postagens do Blog</h1>
  {% for post in posts %}
    <h2>{{ post.title }}</h2>
    <p>{{ post.content }}</p>
    <p><em>Publicado em: {{ post.created_at }}</em></p>
  {% endfor %}
</body>
</html>
```

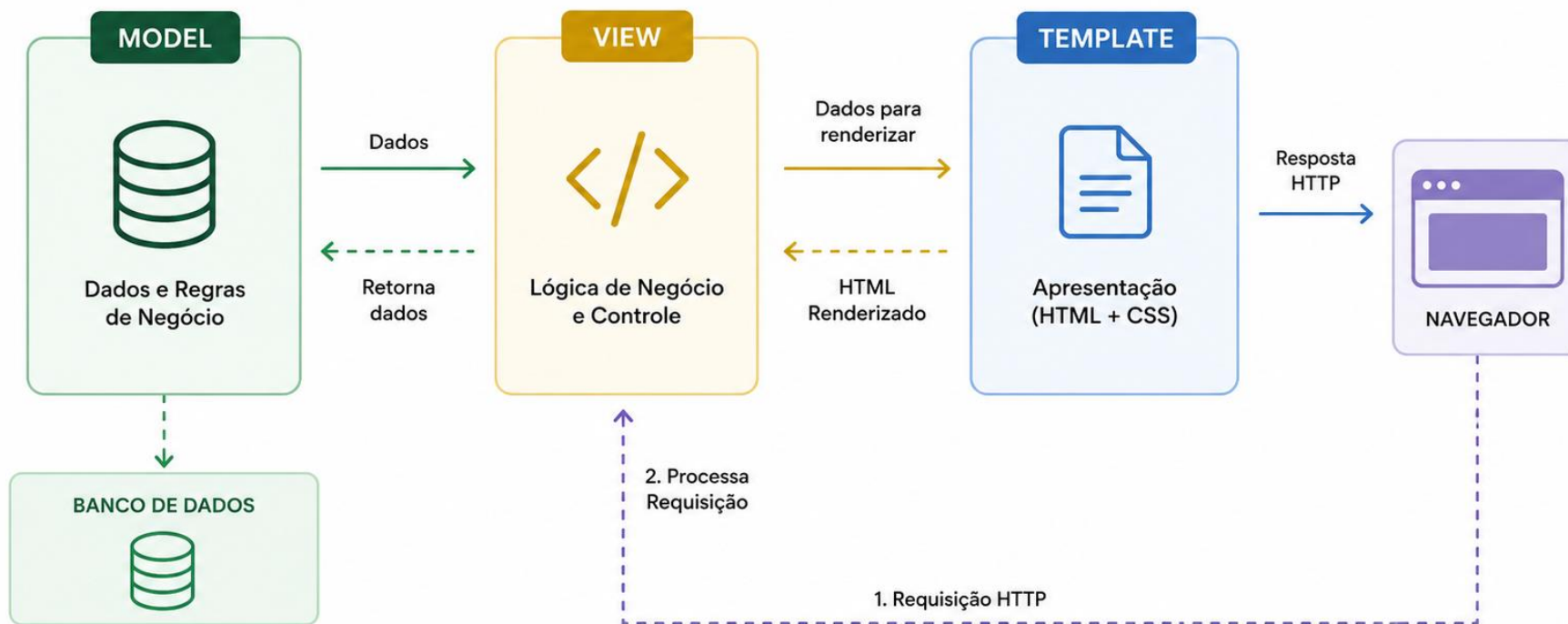
Explicação do Fluxo:

- **Usuário:** O usuário acessa a URL para visualizar as postagens do blog.
- **Requisição HTTP:** A requisição é enviada ao servidor.
- **View (Visão):** A view `post_list` recebe a requisição, consulta as postagens no modelo `Post` e passa os dados para o template `post_list.html`.
- **Template (Modelo de Apresentação):** O template renderiza os dados e cria uma página HTML.
- **Resposta HTTP:** A página HTML é enviada de volta ao navegador do usuário.
- **Usuário:** O usuário vê a lista de postagens do blog no navegador.



django

TEMPLATE (MTV)



O padrão MVT do Django separa claramente as responsabilidades:

- **Model:** Define a estrutura e as regras dos dados.
- **View:** Contém a lógica de negócio e manipula as requisições.
- **Template:** Define como os dados são apresentados aos usuários.

Essa separação facilita o desenvolvimento, a manutenção e a escalabilidade da aplicação.





Preparação do Ambiente



Criar um ambiente virtual

Para criar e ativar um ambiente virtual Python no Windows, execute os seguintes comandos:

```
python -m venv .venv  
.venv\Scripts\activate
```

Caso aconteça um erro ao executar o comando `.venv\Scripts\activate`, você pode habilitar a execução de scripts PowerShell com o comando `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` e tentar novamente.



Instalar o Django

Para instalar o Django, execute o seguinte comando:

```
pip install django
```



Criar um projeto Django

Para criar um projeto Django, execute o seguinte comando:

```
django-admin startproject nome_do_projeto .
```



Criar uma aplicação Django

Para criar uma aplicação Django, execute o seguinte comando:

```
python manage.py startapp nome_da_aplicacao
```



Executar o servidor de desenvolvimento

Para executar o servidor de desenvolvimento do Django, execute o seguinte comando:

```
python manage.py runserver
```





Ola Mundo!



Passo 01:

Abrir o arquivo views.py do app que foi criado Em nosso exemplo foi o lista

Adicionar o seguinte comando dentro deste arquivo:

1. Faça o import da biblioteca HttpResponse
2. Crie uma função chamada home que receba o paramentro request

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("<h1>Ola Mundo!</h1>")
```



Passo 02:

Configurar nosso APP

Agora abra o arquivo settings.py dentro da pasta setup

1. Procurar pela opção: INSTALLED_APPS
2. Adicionar nosso app como na imagem abaixo:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'lista.apps.ListaConfig'  
]
```



Passo 03:

Indicar a rota de acesso, ou seja, informar qual caminho será acessado no navegador para abrir nossa página

1. Abra o arquivo urls.py no setup
2. Fazer o import da nossa função criada no arquivo views: `from fotografia.views import home`
3. Identificar a linha: `urlpatterns`
4. Adicionar a nossa rota: `path("", home)`

Ficará assim

```
from fotografia.views import home

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', home)
]
```



Testando

Abra o endereço: <http://127.0.0.1:8000/> no seu navegador. Se tudo deu certo você verá seu ola mundo!



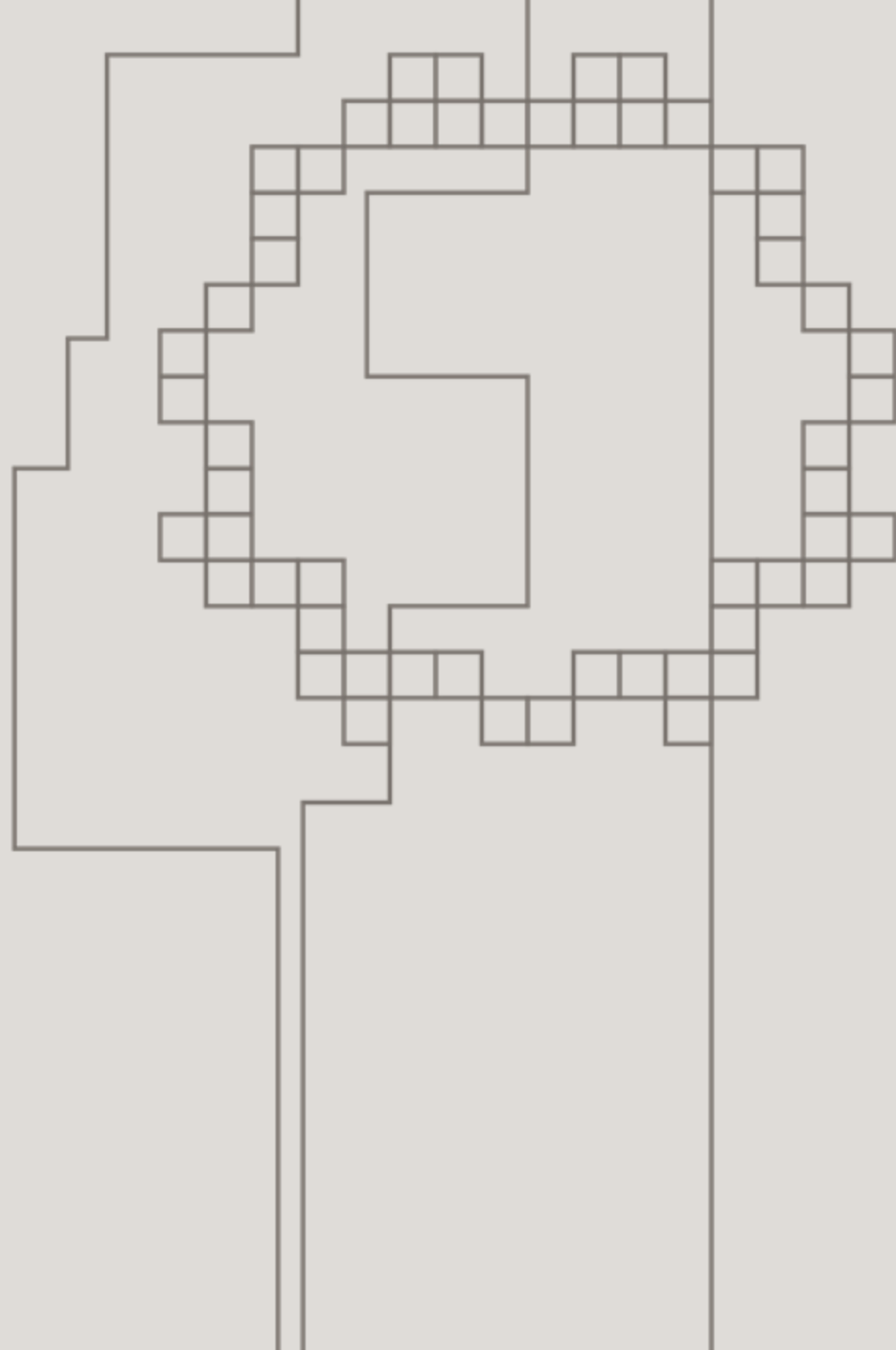


Desafio

Configura uma função chamada contato na views que exiba um formulario com telefone e email na tela

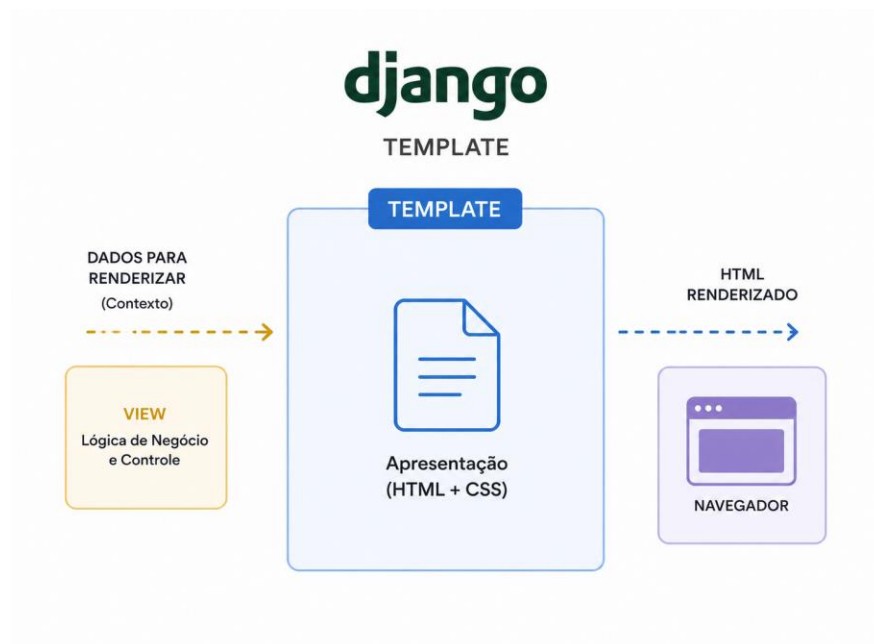


O que é um
template?



O que é um Template no Django?

Um Template no Django é um arquivo que define a estrutura e o layout de uma página web usando HTML e tags específicas do Django para exibir dados dinâmicos.



Passo 01:

1. Criar uma pasta chamada templates dentro da pasta do app

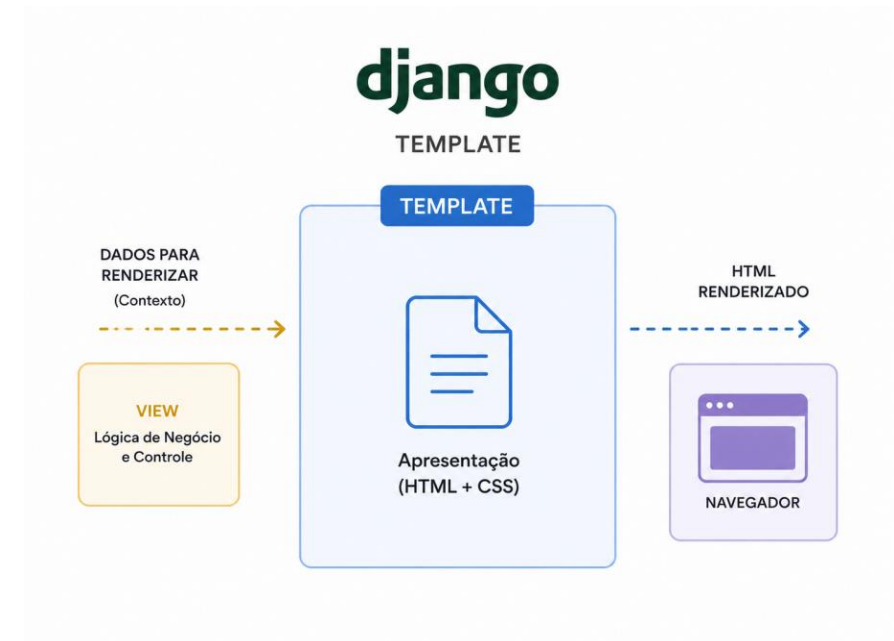
Obs: esta pasta deverá se chamar templates no plural e com letras minúsculas

2. Criar dentro de templates criar uma pasta chamada cliente

3. Criar dentro da pasta criada criar o arquivo home.html

4. Escreva dentro do arquivo home.html a seguinte tag:

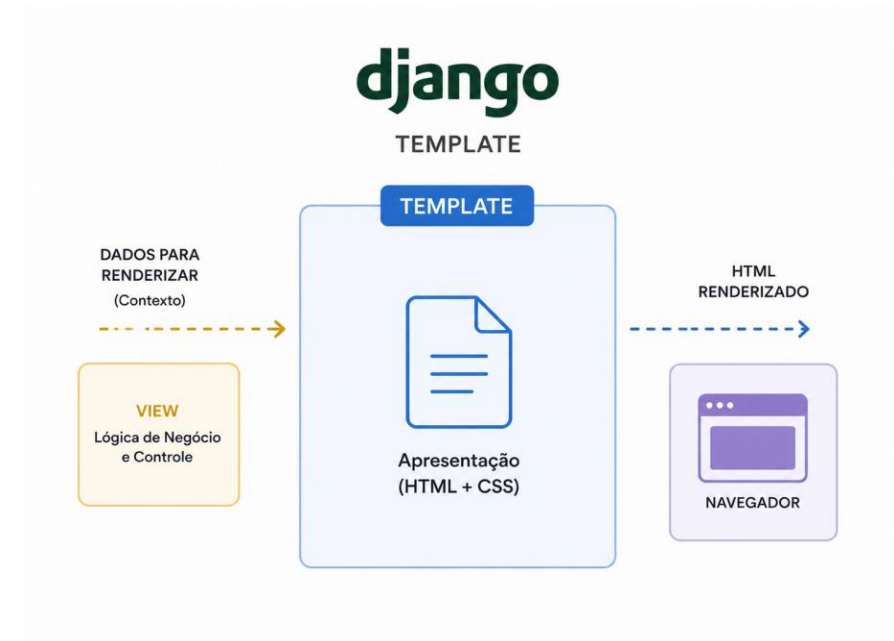
`<h1> Nossos Clientes! </h1>`



Passo 02:

1. Voltar no arquivo views.py dentro do app:
2. Alterar a função home para:

```
def home(request):  
    return render(request, "cliente/home.html")
```





O que nós queremos? Desafios



Volte ao formulario criado anteriormente e configure-o agora com o template

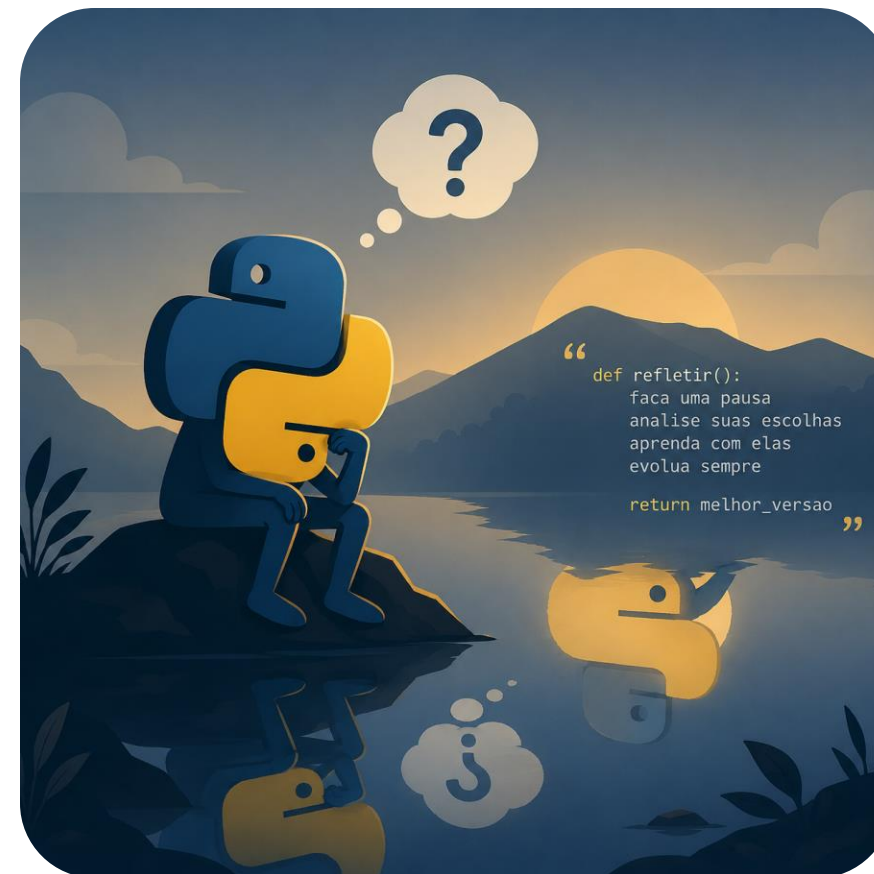




O que é um projeto no Django?

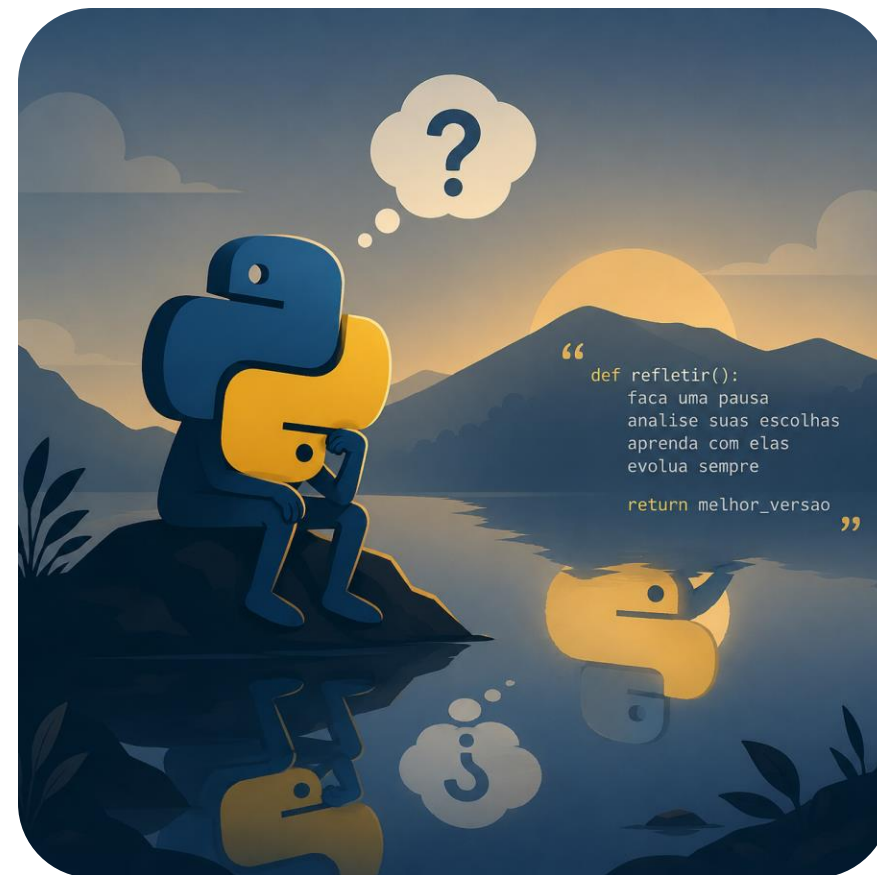


No Django, um projeto é uma coleção de configurações e aplicativos que formam uma aplicação web completa. Ele fornece a estrutura para a construção de um site ou sistema web, incluindo configurações, URLs, aplicativos, modelos de dados, vistas e templates.



Aqui estão os principais componentes de um projeto Django:

1. **Configurações (Settings):** Contêm todas as configurações do projeto, como configurações de banco de dados, locais de templates, entre outras.
2. **URLs:** Um arquivo principal de roteamento (geralmente `urls.py`) que mapeia URLs para as vistas correspondentes.
3. **Aplicativos (Apps):** Um projeto pode ser composto por vários aplicativos. Cada aplicativo é uma funcionalidade ou módulo independente dentro do projeto. Por exemplo, um site de e-commerce pode ter um aplicativo para gestão de produtos, outro para carrinho de compras e outro para processamento de pagamentos.
4. **Modelos (Models):** Definem a estrutura dos dados do projeto e são mapeados para o banco de dados. Eles utilizam o ORM (Object-Relational Mapping) do Django para facilitar as operações com o banco de dados.
5. **Vistas (Views):** Contêm a lógica para processar as requisições e devolver as respostas apropriadas (como HTML, JSON, etc.).
6. **Templates:** Arquivos HTML que definem a apresentação do conteúdo enviado pelas vistas.

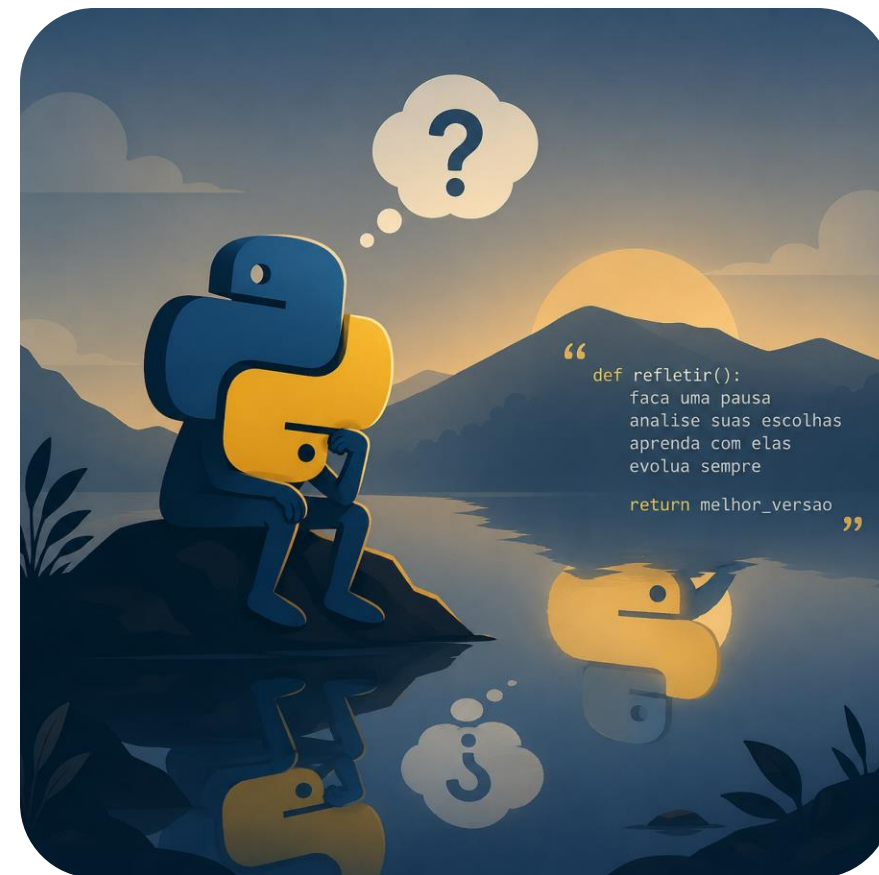


“Em resumo, um projeto Django é a unidade completa que engloba toda a configuração e estrutura necessárias para rodar uma aplicação web. Ele pode ser subdividido em vários aplicativos menores que juntos formam a funcionalidade completa da aplicação.”



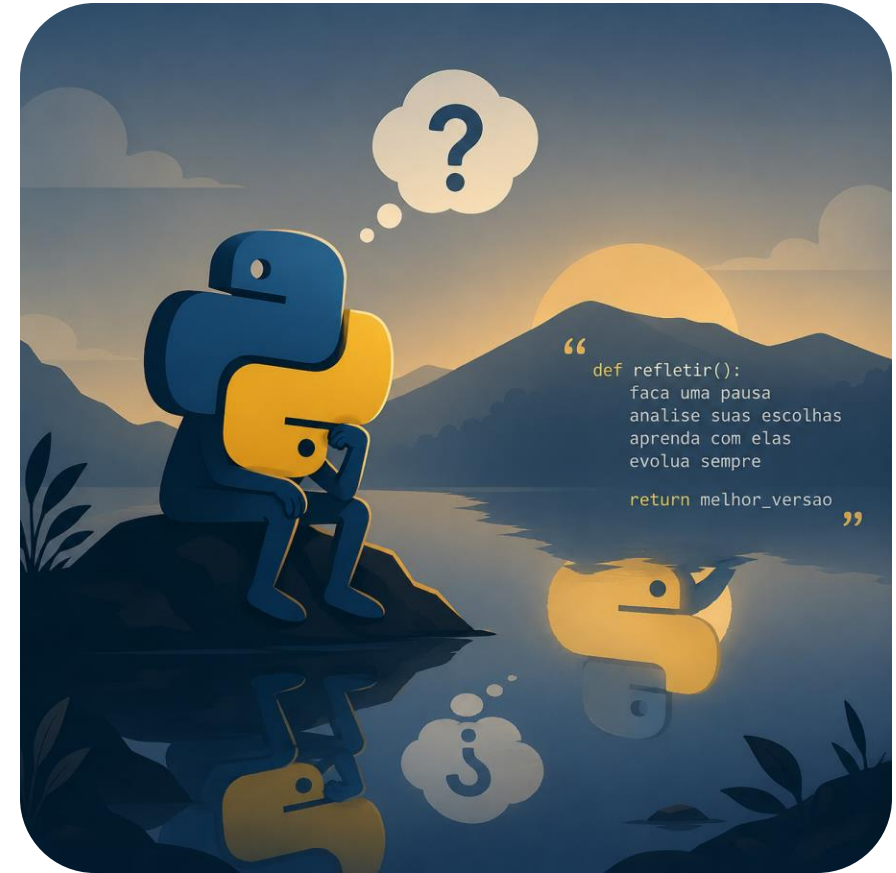
o conceito de "context"

No Django, o conceito de "context" refere-se ao conjunto de dados que você passa para um template ao renderizá-lo. Esses dados são passados na forma de um dicionário, onde as chaves são os nomes das variáveis que você deseja acessar no template e os valores são os dados correspondentes.



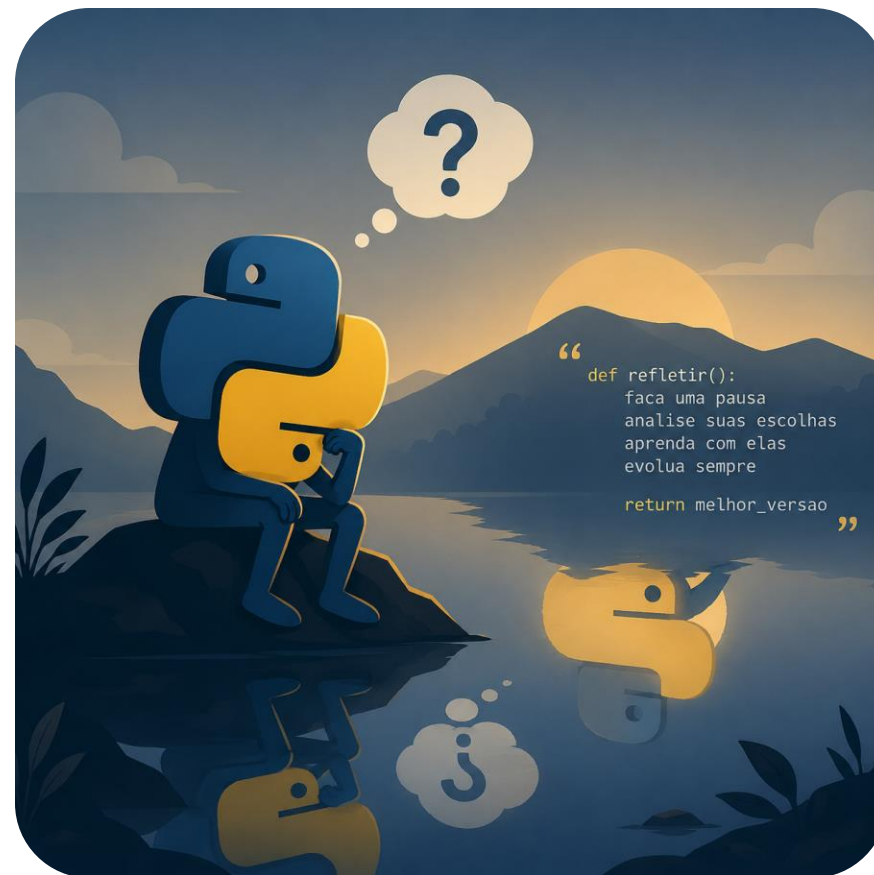
Por que o "context" é Importante?

O contexto é essencial porque ele permite que você passe informações dinâmicas do seu backend para os templates, possibilitando a renderização de conteúdo dinâmico nas suas páginas web. Sem o contexto, os templates do Django seriam estáticos e não poderiam exibir dados específicos para cada solicitação.



Como o "context" Funciona?

- Definindo o "context" na View: Em sua função de view, você cria um dicionário de contexto com as variáveis e valores que deseja passar para o template.
- Passando o "context" para o Template: Ao renderizar o template com a função render, você passa o contexto junto com o nome do template.
- Acessando o Contexto no Template: Dentro do template, você usa a sintaxe {{ variavel }} para acessar e exibir os valores do contexto.



Exemplo Prático

Exibindo uma Variável

Vamos aprender como passar uma variável simples para um template e exibi-la.

```
# views.py
from django.shortcuts import render

def saudacao_view(request):
    mensagem = 'Olá, bem-vindo ao meu site!'
    contexto = {
        'mensagem': mensagem
    }
    return render(request, 'saudacao/saudacao.html', contexto)
```



Criando o Template

```
<!-- saudacao.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Saudação</title>
</head>
<body>
  <h1>{{ mensagem }}</h1>
</body>
</html>
```



Exibindo um Dicionário

Agora, vamos passar um dicionário para o template e exibir seus conteúdos.

Definindo a view

```
# views.py
from django.shortcuts import render

def informacoes_view(request):
    info_dict = {
        'site': 'Meu Site',
        'ano': 2024,
        'autor': 'Seu Nome'
    }
    contexto = {
        'info': info_dict
    }
    return render(request, 'info/informacoes.html', contexto)
```

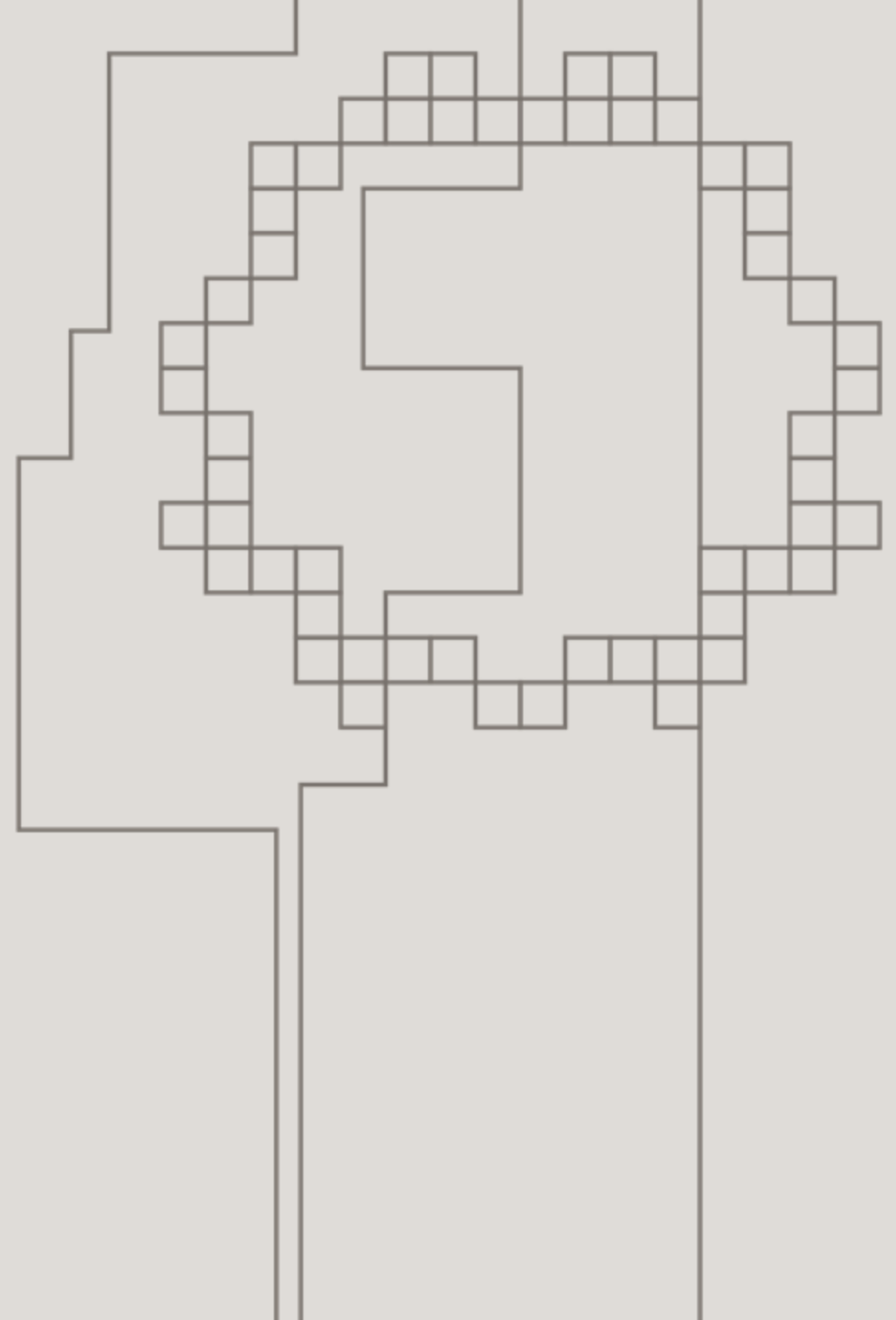


Criando o template

```
<!-- informacoes.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Informações</title>
</head>
<body>
  <h1>Informações do Site</h1>
  <p>Site: {{ info.site }}</p>
  <p>Ano: {{ info.ano }}</p>
  <p>Autor: {{ info.autor }}</p>
</body>
</html>
```



Usando um Loop for
para Iterar uma Lista



Agora, vamos passar uma lista para o template e iterar sobre ela usando um loop for.

Definindo view

```
# views.py
from django.shortcuts import render

def lista_nomes_view(request):
    nomes = ['Maria', 'João', 'Ana', 'Pedro']
    contexto = {
        'nomes': nomes
    }
    return render(request, 'lista_nomes.html', contexto)
```



Criando o template

```
<!-- lista_nomes.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Lista de Nomes</title>
</head>
<body>
  <h1>Lista de Nomes</h1>
  <ul>
    {% for nome in nomes %}
      <li>{{ nome }}</li>
    {% endfor %}
  </ul>
</body>
</html>
```



Combinando Variável, Dicionário e Loop for

Definindo a View

```
# views.py
from django.shortcuts import render

def exemplo_view(request):
    mensagem = 'Bem-vindo ao meu site!'
    lista_nomes = ['Maria', 'João', 'Ana', 'Pedro']
    info_dict = {
        'site': 'Meu Site',
        'ano': 2024,
        'autor': 'Seu Nome'
    }
    contexto = {
        'mensagem': mensagem,
        'nomes': lista_nomes,
        'info': info_dict
    }
    return render(request, 'exemplo.html', contexto)
```



Criando o Template

```
<!-- exemplo.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Exemplo</title>
</head>
<body>
  <h1>{{ mensagem }}</h1>

  <h2>Lista de Nomes:</h2>
  <ul>
    {% for nome in nomes %}
      <li>{{ nome }}</li>
    {% endfor %}
  </ul>

  <h2>Informações:</h2>
  <p>Site: {{ info.site }}</p>
  <p>Ano: {{ info.ano }}</p>
  <p>Autor: {{ info.autor }}</p>
</body>
</html>
```





Desafio Final. Aff

Desafio:

Você foi selecionado pela empresa XPTO Tecnologias para uma entrevista de emprego. A XPTO trabalha com desenvolvimento de aplicações Web e, devido a flexibilidade e escalabilidade o Django é o framework principal no dia a dia da empresa. A sua avaliação de emprego será pautada em desenvolver uma aplicação de cadastro de alunos para um cliente da XPTO do ramo escolar. Requisitos da aplicação:

1. Criar uma nova pasta e chama-la de xpto (area de trabalho)
2. Abrir esta pasta no vscode
3. Criar um novo ambiente virtual
4. Instalar o django
5. Criar um novo projeto em django chamado tuxschool
6. Criar um app chamado cadastro
7. No template deste app deverá conter uma lista com nome, curso, turma e dois botões: um chamado edição e o outro chamado excluir para cada aluno

OBS: Os dados dos alunos deverão vir de uma lista de dicionário contida na view e deverão ser exibidas no template através do conceito de context do django.

ATÉ A
PRÓXIMA
AULA!